

Artifacts & Reproducibility

This section records the concrete artifacts that make the book’s claims inspectable, rebuildable, and testable. The intent is not merely archival: each item below is part of the reproducibility protocol for the technical development in the main text.

- **Manifest.** A manifest enumerates the code and data used in the project together with stable hashes. It should identify the relevant source files, generated outputs, and any auxiliary datasets or tables needed to reproduce a result. Where possible, the manifest should also record version identifiers for external dependencies so that a build can be reconstructed from the recorded state.
- **Build scripts.** Build scripts encode the steps required to transform source material into the published artifacts. These scripts should be deterministic, machine-readable, and sufficient to regenerate the book’s outputs from the manifest alone, subject to the stated environment assumptions.
- **Containers.** Container definitions capture the execution environment used for builds and checks. By pinning toolchain versions and system dependencies, containers reduce ambiguity about compiler behavior, library availability, and platform-specific differences.
- **Harnesses.** Test harnesses specify the checks used to validate claims, examples, and derived outputs. A harness may include unit tests, property tests, regression tests, or proof-checking scripts, together with short descriptions of what each test is intended to establish.
- **CI configuration.** Continuous integration configuration records how the build and test harnesses are executed automatically. This includes the order of checks, the environment in which they run, and the conditions under which a build is considered successful.
- **Outputs.** Outputs are the generated artifacts produced by the build and verification pipeline. These may include compiled documents, tables, logs, figures, and other derived files that substantiate the claims made in the text.

For each artifact class, the reproducibility standard is the same: the artifact should be identifiable, versioned, and linked to the procedure that produced it. When a result depends on a nontrivial computation or transformation, the corresponding manifest entry, script, and output should be kept together so that the chain from source to result remains auditable.

In practice, the reproducibility package for this book consists of the following minimal set:

1. a manifest of code and data with hashes;
2. the build scripts used to generate the book and its derived materials;
3. container definitions or equivalent environment specifications;
4. harnesses describing tests and verification steps;
5. CI configuration for automated execution;
6. the resulting outputs and logs.

Together, these artifacts support the book’s claims-as-builds methodology: every substantive claim should be traceable to a reproducible procedure, and every procedure should leave behind an inspectable output.

To make the protocol operational, the repository should keep the manifest, scripts, containers, harnesses, CI configuration, and outputs in a single auditable chain. A minimal implementation can be organized as follows:

- a top-level manifest that lists all tracked source files, generated files, and external inputs with hashes;
- one or more build scripts that regenerate the book and any derived tables, figures, or logs;
- a container or equivalent environment specification that pins the toolchain;
- test harnesses that describe the checks performed on claims, examples, and derived results;
- CI configuration that runs the build and tests in a fixed order;
- archived outputs that record the exact products of the pipeline.

This arrangement is intentionally conservative. It does not require every artifact to be self-explanatory in isolation; rather, it requires that each artifact be linked to the others in a way that permits inspection, rerun, and comparison against the recorded hashes. In that sense, reproducibility is not a single file or script but a disciplined relation among source, environment, procedure, and output.

For operational clarity, the repository should treat the following as the canonical reproducibility bundle:

- **manifest** files that enumerate inputs, outputs, and hashes;
- **build** scripts that regenerate the published materials;
- **container** or environment specifications that pin the toolchain;
- **harness** files that describe tests, checks, and expected outcomes;
- **ci** configuration that automates the build-and-test sequence;
- **outputs** and logs that record the exact results of the pipeline.

The practical criterion is simple: if a reader can start from the manifest, follow the recorded procedure, and recover the archived outputs under the stated environment, then the artifact set is sufficiently reproducible for the purposes of this book.

At a minimum, the reproducibility bundle should make the following items easy to locate and compare:

- the exact source snapshot used for the build;
- the hashes of generated files and external inputs;
- the scripts that transform source into output;
- the container image or environment specification used during execution;
- the test descriptions and CI job definitions;

- the archived outputs, logs, and any failure reports.

This section is therefore a checklist as much as a description. It names the artifacts that must exist, the relationships among them, and the evidence required to treat the book’s technical claims as reproducible builds rather than isolated assertions.